



Fundamentos de IoT y Programación Aplicada con Python

Preparado por: Mgtr. Tomás Calderón

1. Las colecciones nativas estructuran secuencias de datos físicas

Listas vs. Tuplas en Python

Listas: Estructuras Mutables

Son arreglos dinámicos en el Heap de memoria. Permiten almacenar elementos de cualquier tipo y cambiar su longitud dinámicamente mediante operaciones que alteran su estado físico.

Tuplas: Estructuras Inmutables

Una vez creadas en memoria, su longitud y referencias internas son inmutables de solo lectura. Son altamente eficientes en el uso de memoria RAM por no requerir sobreasignación (*over-allocation*).



```
# Modificando un histórico de telemetría
lecturas = [24.5, 26.1, 23.9]
lecturas.append(25.0) # Agrega al final
lecturas.remove(24.5) # Elimina la primera coincidencia
print(lecturas)      # [26.1, 23.9, 25.0]

# Configuración fija de pines (Inmutable)
config_pines = (4, 5) # Pines de la placa
# config_pines[0] = 12 # Provocaría un TypeError!
```

💡 RETO EXPRESS: Inmutabilidad física

¿Si recibiremos lectura de un dispositivo IoT sobre su ubicación geográfica fija; estas dadas en Latitud y longitud, es mejor utilizar una Lista o una Tupla?

2. Los diccionarios y conjuntos aceleran la búsqueda indexada

Diccionarios y Conjuntos(set)

- **Los Diccionarios (dict)** mapean pares clave-valor. Python los implementa como tablas hash de alta velocidad con búsquedas $O(1)$. Claves obligatoriamente inmutables (ej: strings, enteros, tuplas). siendo la estructura perfecta para serializar y estructurar datos en formato JSON.

- **Los Conjuntos (set)** son colecciones desordenadas de elementos hashables únicos. No se permiten duplicados. Son excelentes para filtrado rápido de IDs de dispositivos redundantes.

💡 RETO EXPRESS: Claves

¿Es posible utilizar una lista de números [1, 2] como clave de un diccionario?



```
# Empaquetando telemetría estructurada en diccionario
payload = {
    'sensor_id': 'DHT22_01',
    'temp': 25.4,
    'hum': 60.2
}
print(payload['temp']) # Acceso directo a la clave
```

```
# Limpiando una lista de lecturas de sensores con valores
repetidos lecturas_lista = [22.5, 23.0, 22.5, 24.1, 23.0]
```

```
# Al convertirla a set, los duplicados desaparecen
lecturas_unicas = set(lecturas_lista)
```

```
# Resultado: {24.1, 22.5, 23.0} (El orden no está garantizado)
print(lecturas_unicas)
```

3. El recorrido iterativo organiza colas temporales en tiempo real

Optimización de Memoria en IoT

En sistemas embebidos, la RAM es un recurso extremadamente limitado. Por lo tanto, hay que considerar como prioritario desarrollar las opciones que en ejecución sean las más eficientes.

Iteración Eficiente y Buffers

- El método `.items()` de un diccionario permite iterar sobre las claves y sus valores asociados en un solo paso de manera eficiente.
- La organización de datos temporales requiere estructuras de cola (FIFO). Podemos implementar un buffer circular acotado usando listas.
- Al usar `pop(0)` removemos el elemento más antiguo, permitiendo calcular promedios móviles sin agotar los recursos del Heap del sistema.



```
# Buffer circular de lecturas temporales
buffer = []
max_capacidad = 3
nuevas_lecturas = [22.1, 23.4, 21.9, 24.5]

for lectura in nuevas_lecturas:
    buffer.append(lectura)
    if len(buffer) > max_capacidad:
        buffer.pop(0) # Saca el más antiguo
    print('Buffer actual:', buffer)
# Salida final: [23.4, 21.9, 24.5]
```

💡 RETO EXPRESS: ¿Es Eficiente?

Teniendo en cuenta que el `pop(0)`, al eliminar el elemento del arreglo; internamente python acomoda las posiciones del resto de los elementos. ¿Es lo más eficiente esto?, ¿Se puede mejorar?

4. Las funciones

Modularidad Nativa

- Las funciones en Python se declaran con 'def'. No requieren llave de cierre ni llaves de bloque; todo se rige bajo la indentación de 4 espacios. Pueden retornar valores con la palabra reservada 'return'.
- Type Hints opcionales: Añaden legibilidad y documentación estática sobre los tipos de parámetros y el tipo del valor retornado.
- Parámetros por defecto: Permiten definir configuraciones por omisión (ej: factor de calibración de un transductor).
- Retorno múltiple: Python empaqueta automáticamente múltiples valores separados por coma en una tupla compacta de lectura rápida.

```
# Función con anotación de tipos
def calcular_promedio(lecturas: list) -> float:
    if not lecturas:
        return 0.0
    return sum(lecturas) / len(lecturas)

# Función con parámetros por defecto
def leer_sensor(pin: int, offset: float = 0.0):
    valor_crudo = 23.4 # Valor simulado
    return valor_crudo + offset

# Invocar usando argumentos posicionales y nombrados
p = calcular_promedio([22, 24, 26])
r = leer_sensor(pin=4, offset=1.5)
```

💡 RETO EXPRESS: Argumentos

Al declarar una función que recibe parámetros, bajo qué circunstancias especificaría el tipo de dato que recibe, y bajo cuales no se haría?

5. Organización Modular

Namespaces y Reutilización

- Un Módulo es un simple archivo con extensión .py. Permite encapsular variables, clases e identificadores de funciones.
- Un Paquete es un directorio que contiene múltiples módulos y un archivo especial de inicialización (__init__.py).
- Sintaxis 'import': Incorpora el espacio de nombres completo. Requiere invocar los objetos con el prefijo del módulo.
- Sintaxis 'from ... import': Extrae un componente directamente al namespace local. Es ideal para optimizar memoria en entornos embebidos.



```
# Archivo 1: calibrador.py
FACTOR_CALIBRACION = 1.05
def calibrar_senal(v: float) -> float:
    return v * FACTOR_CALIBRACION

# Archivo 2: main.py (Importación selectiva)
from calibrador import calibrar_senal
resultado = calibrar_senal(12.0)
print(resultado) # 12.6
```

💡 RETO EXPRESS: Colisión de Nombres

Si en un archivo python A se tiene un import de otro Python B, y luego localmente en el python A se declara una función con el mismo nombre de una función ya existente en el python B, ¿qué sucede?

6. Bibliotecas Externas: Gestión con pip

El Gestor de Paquetes (pip)

Python Package Index (PyPI) aloja miles de bibliotecas externas creadas por la comunidad. El comando 'pip install' automatiza la descarga y resolución de dependencias del proyecto.

Inspección Fuera de Línea

Python incluye herramientas integradas en su consola interactiva (REPL).
`dir(objeto)` lista todos los métodos y atributos disponibles en memoria, y
`help(objeto)` muestra la documentación técnica del fabricante.

```

# Comandos ejecutados en consola del sistema
$ pip install paho-mqtt

# Exploración interactiva REPL en Thonny
>>> import math
>>> print(dir (math)) # Lista todos los atributos del módulo
['acos', 'acosh', 'asin', ...]

>>> help(math.sin) # Muestra el docstring y firma
sin(x, /) -- Return the sine of x (measured in radians).
```

💡 RETO EXPRESS: Exploración matemática

Utilizando el código de arriba, cómo imprimiría el seno de pi medio.

7. Entendiendo el Hardware

Computadora Personal (PC)

- Cuenta con Sistema Operativo (Windows/Linux) complejo.
- Gigabytes de Memoria RAM (giga-escala) y CPU multi-núcleo a GHz.
- El software interactúa con abstracciones (APIs de SO, controladores).
- Diseñada para procesamiento de propósito general.

Microcontrolador (MCU)

- No tiene Sistema Operativo (corre el código directamente *bare-metal*).
- Recursos ultra limitados: Kilobytes (KB) de RAM, Megabytes (MB) de Flash.
- CPU mononúcleo a MHz (ej. 80-240 MHz).
- Interactúa con el mundo real mediante Pines Eléctricos (GPIO).

Pines GPIO (General Purpose I/O)

Son las terminales físicas (patitas metálicas) de un chip. El programador puede controlar su estado eléctrico para leer el mundo físico o interactuar con él mediante voltajes discretos o señales de buses serie (I2C, SPI).

⚡ RETO EXPRESS:

Si un microcontrolador como la Raspberry Pi Pico tiene tan solo 264 KB de RAM disponible, ¿cuál es el peligro latente de escribir un programa en Python que no sea eficiente?

8. MicroPython: El Intérprete Reducido

Python en el Metal (Bare-Metal)

MicroPython es una implementación de Python 3 diseñada específicamente para microcontroladores. Se ejecuta directamente sobre el procesador de silicio sin necesidad de un sistema operativo subyacente.

Control Manual de Memoria (gc)

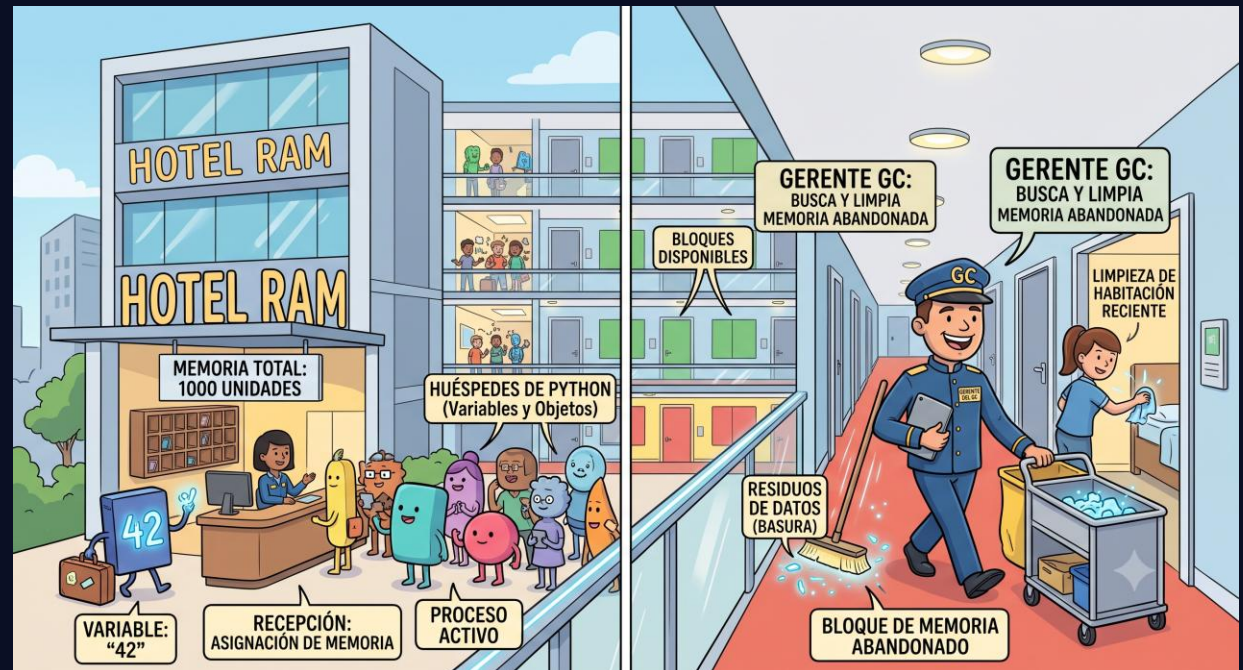
Para evitar la fragmentación de la memoria Heap sobre chips limitados, MicroPython cuenta con un recolector de basura (`gc`) optimizado. Podemos invocar `gc.collect()` manualmente para forzar la liberación de variables descartadas.

Ejemplo:

Supongamos que la RAM es un hotel.

Los procesos, variables, etc, que crea Python son los huéspedes del hotel.

El gc es el administrador del Hotel.



9. El arranque de MicroPython implementa un flujo secuencial estricto

1. El Arranque: `boot.py`

Al energizar o resetear el chip, MicroPython arranca y ejecuta primero el script `boot.py`. Aquí se inicializan las configuraciones globales fijas de hardware de bajo nivel por ejemplo:

- Montaje de sistemas de archivos si el proyecto necesita almacenar lecturas de sensores.
- Preparar la memoria externa del dispositivo si es el caso.
- Iniciar el estado y calibrar los GPIO para reducir comportamientos erráticos.

2. La Aplicación: `main.py`

Finalizado `boot.py`, se ejecuta automáticamente `main.py`. Este archivo contiene la lógica de la aplicación. Debe correr en un bucle infinito para que el chip nunca deje de operar.

En resumen,

El ciclo de vida en estos entornos embebidos es una transición de dos pasos: primero el chip prepara su entorno físico (`boot.py`) e inmediatamente después entra en un estado de monitoreo y procesamiento infinito de su lógica de negocio (`main.py`).

9. Ejemplos en código:

```
#Código resumido para iniciar la interfaz de red desde el boot.py
import network
import time

print("--- Iniciando secuencia de arranque (boot.py) ---")
print("Configurando hardware y levantando red...")

while not wifi.isconnected():
    print("Conectando a la red...")
    time.sleep(1)

print("¡Conexión Wi-Fi exitosa!")
```

```
# Código resumido de un main.py
import time

print('Iniciando ciclo de monitoreo principal...')

while True:
    # 1. Leer sensor físico
    print('Capturando lectura...')
    # 2. Procesar y transmitir
    time.sleep(5) # Suspensión temporal en segundos
```

10. La biblioteca 'machine' e interacción con GPIO

La Biblioteca Nativa 'machine'

Es el módulo estándar de MicroPython para interactuar directamente con los periféricos de silicio del microcontrolador. Nos permite abstraer la manipulación de registros lógicos a clases de Python.

La Clase Pin

Permite configurar un pin GPIO físico como ****Entrada**** (`Pin.IN`) para leer sensores digitales, o como ****Salida**** (`Pin.OUT`) para encender actuadores, enviando voltajes altos (3.3V) o bajos (0V).



```
from machine import Pin
```

```
# 1. Configuramos el hardware
```

```
sensor_puerta = Pin(14, Pin.IN) # Lee datos del exterior
```

```
alarma = Pin(15, Pin.OUT)      # Envía energía al exterior
```

```
# 2. La lógica de negocio
```

```
estado_puerta = sensor_puerta.value()
```

```
if estado_puerta == 1: # Envía 3.3V físicos por el pin 15
```

```
    alarma.value(1)
```

```
else: # Envía 0V físicos por el pin 15
```

```
    alarma.value(0)
```

MICROPYTHON GPIO: CÓDIGO-A-HARDWARE (Inputs y Outputs)

ENTRADA DIGITAL (Input)

CODIGO MICROPYTHON

```
from machine import Pin
import time

# Configuración del Pin 14 como entrada
boton = Pin(14, Pin.IN, Pin.PULL_UP)

# Bucle principal para leer
while True:
    estado = boton.value()
    print("Estado del boton:", estado)
    time.sleep(0.1)
```

ENTRADA
DIGITAL
(INPUT)

'1' (no presionado)
'0' (presionado)

FLUJO FÍSICO

ESP32 / Pico

INTERNAL
PULL-UP

ESP32 / Pico

SALIDA DIGITAL (Output)

CODIGO MICROPYTHON

```
from machine import Pin
import time

# Configuración del Pin 15 como salida
led = Pin(15, Pin.OUT)

# Bucle principal para encender/apagar
while True:
    led.on() # Pin 15 = 3.3V
    time.sleep(1)
    led.off() # Pin 15 = 0V
    time.sleep(1)
```

Pin 15
voltaje

Time

ENCENDIDO

APAGADO

VOLTAJE
FÍSICO
3.3V

VOLTAJE
FÍSICO
0V

FLUJO
FÍSICO

CONCEPTOS CLAVE: `Pin`, `machine`, `IN`, `OUT`, `PULL\_UP`, `value()`, `on()`, `off()`

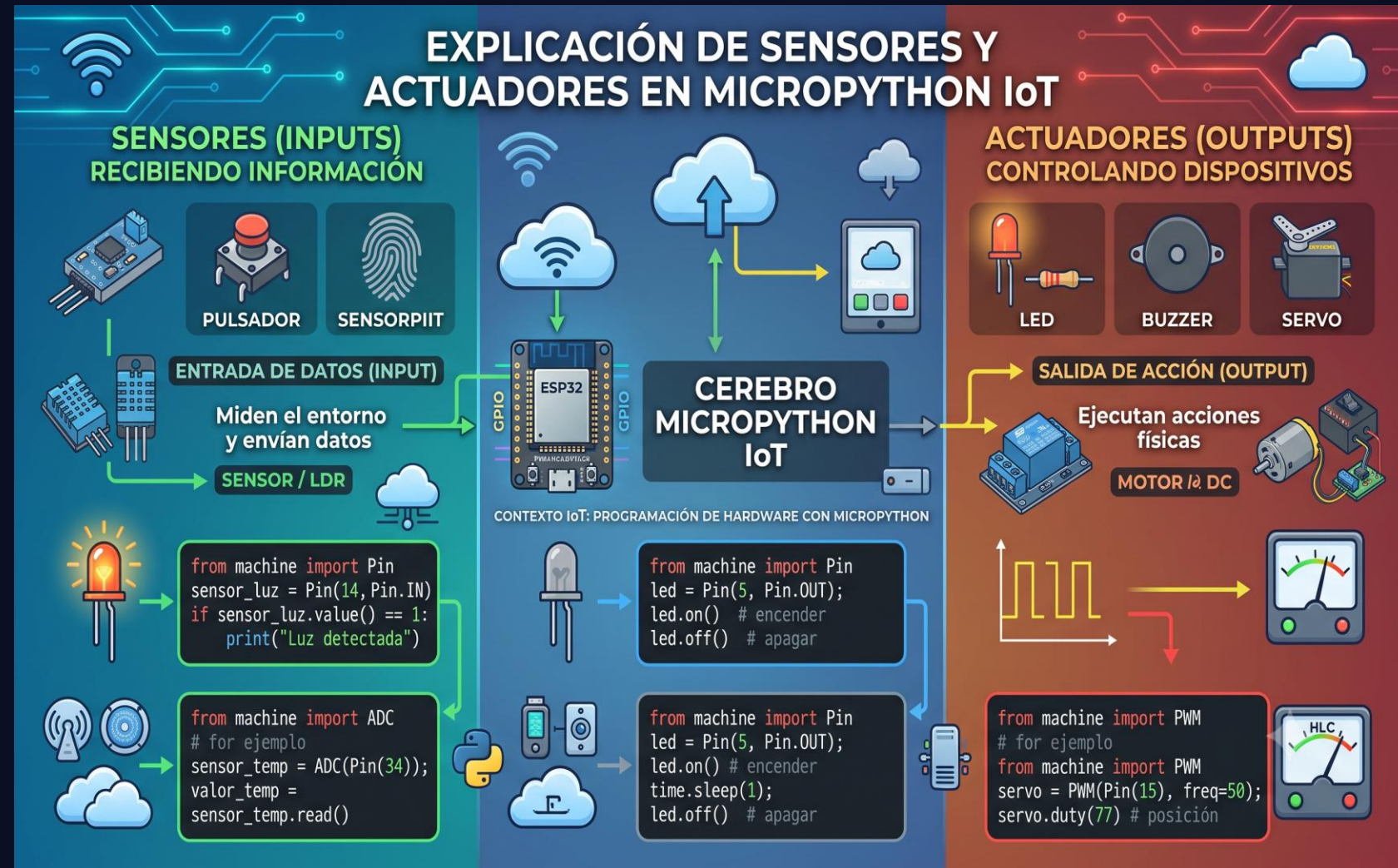
11. Sensores y Actuadores

Sensores

son dispositivos de entrada que se encargan de adquirir y capturar datos del entorno. Su función es traducir fenómenos físicos (como la temperatura, la humedad, la luz o la interacción mecánica) en señales eléctricas que el microcontrolador y el software pueden interpretar.

Actuadores

Los actuadores son dispositivos de salida que ejecutan una acción en el mundo físico basándose en las órdenes que reciben del microcontrolador. Su función es inversa a la del sensor: toman las señales eléctricas generadas por el software y las transforman en energía mecánica, lumínica o térmica.

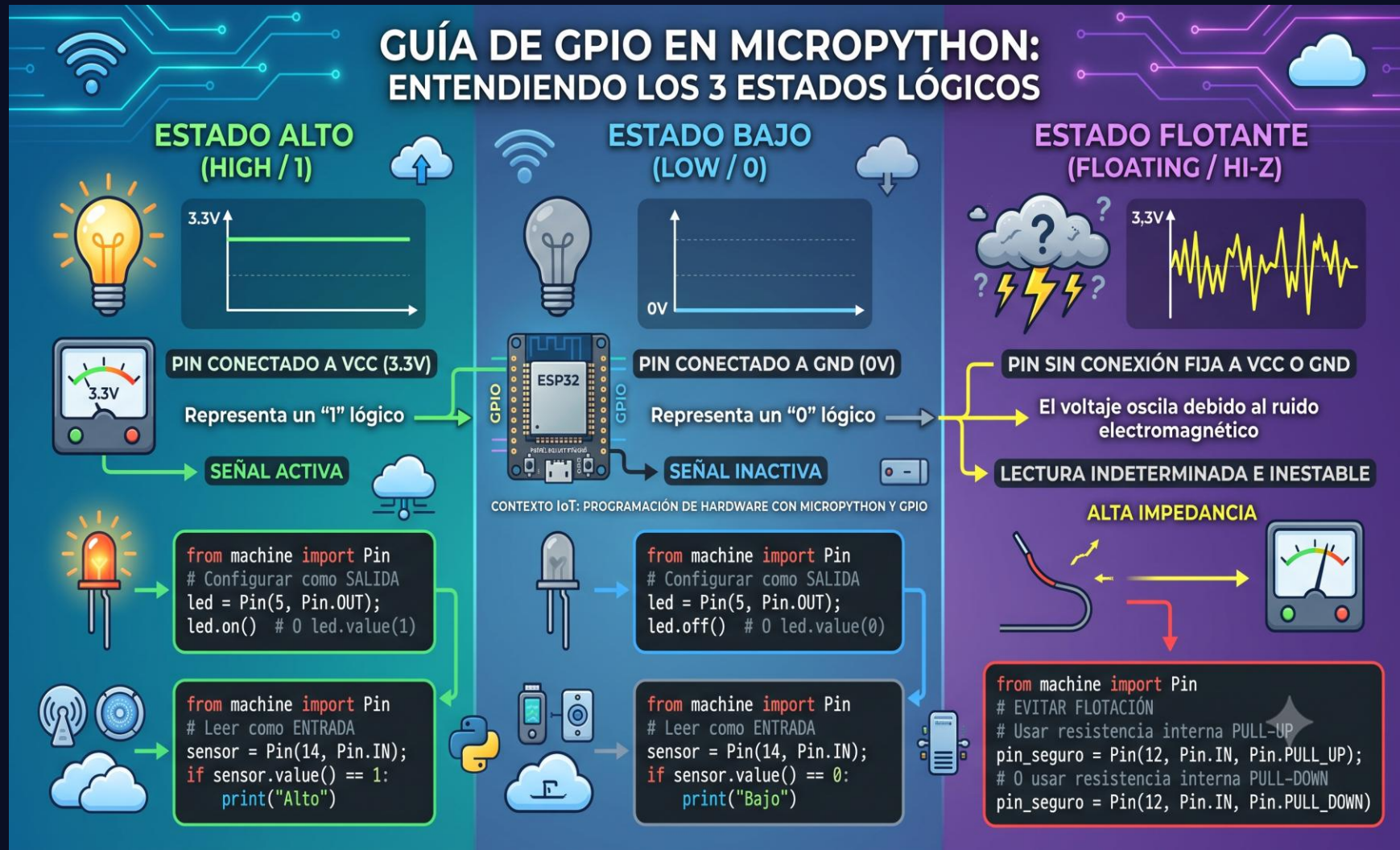


12. Cómo leer sensores todo/nada y evitar estados eléctricos flotantes usando Pull-Up

El Estado Flotante en Silicio

Si usted configura un pin como entrada (Pin.IN) para leer un botón, pero el botón no está presionado, el pin queda físicamente desconectado al aire. En este escenario, el pin se convierte en una antena microscópica que absorbe ruido electromagnético del ambiente.

A este estado inestable se le llama estado flotante. Si el software hace una lectura digital en ese momento, leerá de forma caótica y aleatoria una mezcla de ceros y unos, haciendo que el sistema crea que el botón se está presionando.



12. Cómo leer sensores todo/nada y evitar estados eléctricos flotantes usando Pull-Up

Prevenir el Estado Flotante

Para prevenir el estado flotante mediante código en MicroPython, se debe utilizar la configuración de pines GPIO de la biblioteca machine.

La solución se aplica en el momento exacto en que usted declara la variable del sensor o botón, ordenándole al microcontrolador que active sus resistencias internas mediante el parámetro `Pin.PULL_UP` (o `Pin.PULL_DOWN`, dependiendo del diseño del circuito).



```
from machine import Pin
import time

# Entrada con Pull-Up interno activado
boton = Pin(4, Pin.IN, Pin.PULL_UP)

while True:
    # Si se presiona el botón, se conecta a GND
    if boton.value() == 0:
        print('¡Botón Presionado!')

    time.sleep(0.1) # Evita lecturas a alta velocidad
```

13. Temporización (Timing)

La importancia del 'Timing'

Un microcontrolador es tan rápido que, si no le indicamos cómo manejar el tiempo, ejecutaría las lecturas de los sensores millones de veces por segundo, saturando el procesador y consumiendo exceso de energía. La temporización es la técnica de controlar "cuándo" y "con qué frecuencia" ocurren las cosas en el código.

Delays: Es el uso de funciones de pausa para darle respiros al procesador y crear ciclos de monitoreo eficientes.

Timers: Son herramientas de programación orientada a eventos que permiten que el microcontrolador ejecute una tarea periódica de forma automática (por ejemplo, "lee la humedad exactamente cada 10 minutos") sin necesidad de detener o bloquear el resto del programa.

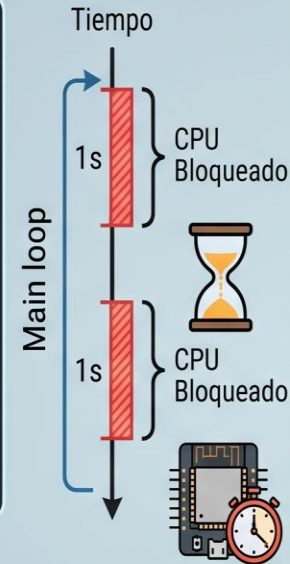
TEMPORIZACIÓN EN MICROPYTHON PARA IoT

DELAYS: ESPERA ACTIVA (Bloqueante)

```
import time
from machine import Pin

led = Pin(2, Pin.OUT)

# Bucle Principal (Wait active)
while True:
    led.on()
    time.sleep(1) # CPU Bloqueado por 1s
    led.off()
    time.sleep(1) # CPU Bloqueado por 1s
```



Detiene el programa principal por un tiempo determinado. Simple pero ineficiente para multitarea.

Ejemplo: Solo parpadeo



TIMERS: GESTIÓN ASÍNCRONA (No bloqueante)

```
from machine import Timer, Pin

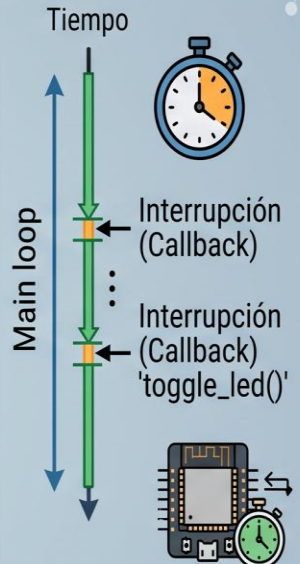
led = Pin(2, Pin.OUT)

# Función de retorno (Callback)
def toggle_led(timer):
    led.value(not led.value())

# Crear Temporizador de Hardware
tim = Timer(-1)

# Configurar y arrancar
# period: ms, mode: PERIODIC, callback
tim.init(period=1000,
        mode=Timer.PERIODIC,
        callback=toggle_led)

# El programa principal sigue corriendo
# while True: pass # (Otras tareas)
```



Usa el temporizador de hardware. Ejecuta una función cuando el tiempo expira. Eficiente para multitarea.

Ejemplo IoT: Parpadeo LED + Leer Sensor + Enviar Nube

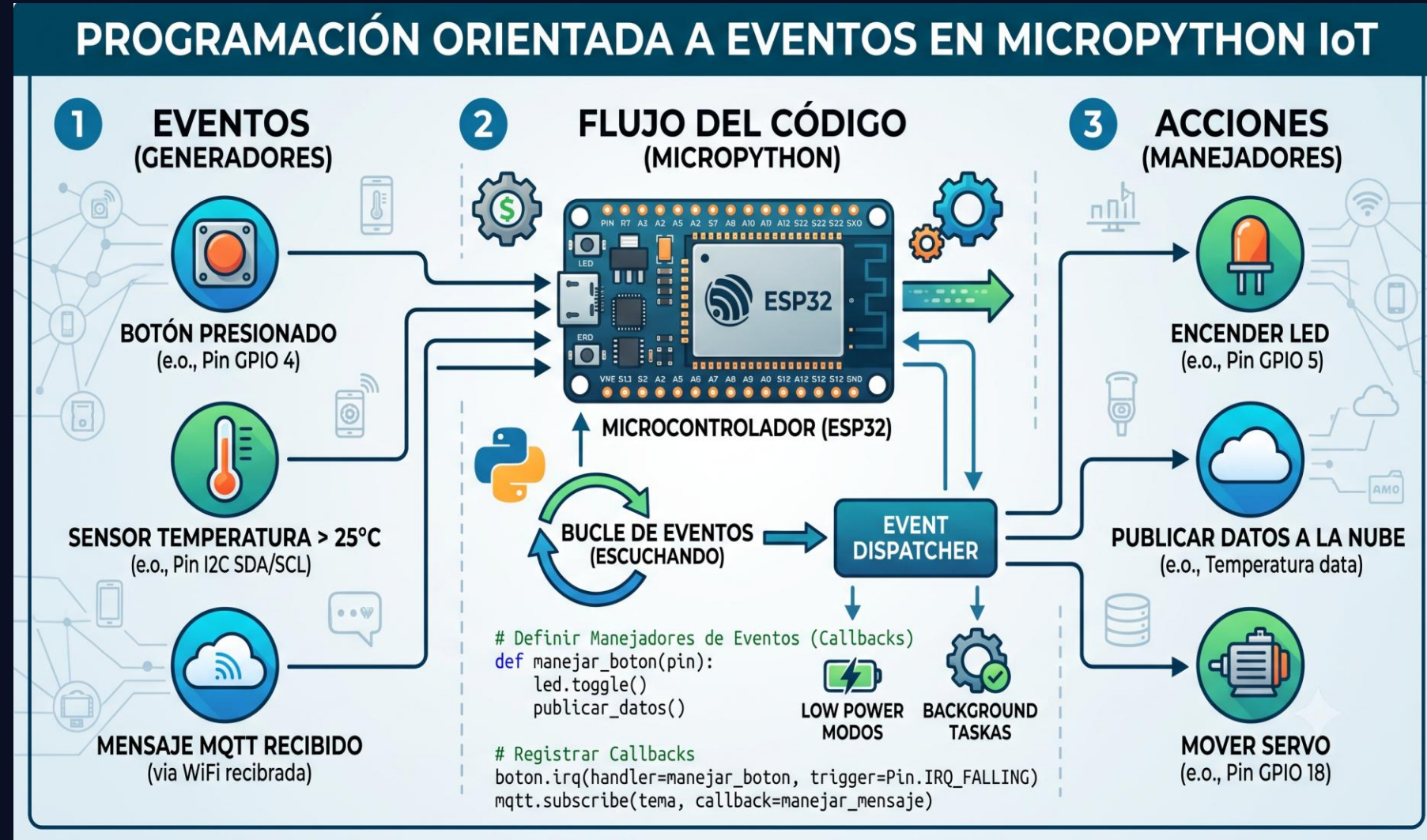


14. Programación Orientada a Eventos

Programación Reactiva

Es un paradigma donde el flujo de ejecución del programa no sigue una secuencia lineal estricta, sino que está determinado por sucesos o "estímulos" externos.

En lugar de que el microcontrolador esté constantemente preguntándole a un sensor "¿Ya cambiaste de valor? ¿Ya cambiaste de valor?" (una técnica ineficiente conocida como polling que desperdicia ciclos de CPU y batería), el sistema simplemente "se va a dormir" o continúa con su ciclo de monitoreo principal, y solo reacciona cuando el sensor le "avisa" que algo ocurrió.



15. Funciones para manejar eventos

El Evento Físico: `Pin.irq()` (Interrupciones de Hardware)

Las interrupciones son la capacidad técnica que tiene el hardware del microcontrolador para recibir una alerta física (como la detección de movimiento en un sensor) e interrumpir momentáneamente lo que el procesador estaba haciendo para atender esa urgencia.

Para implementar el manejo de interrupciones a nivel de código, se utiliza el método `.irq()` (Interrupt Request) nativo de la clase `Pin`.

El proceso a nivel de código se divide estrictamente en tres pasos: crear la función manejadora, configurar el pin físico y vincular ambos mundos.

Reglas para manejar interrupciones:

- **Breves y Rápidas:** Deben ejecutar el mínimo código posible y retornar rápidamente al programa principal.
- **No Asignar Memoria:** Evita crear nuevos objetos (listas, diccionarios, etc.) dentro del manejador.
- **Evitar Funciones Complejas:** No uses `print()` o funciones de temporización largas como `time.sleep()`.

15. Funciones para manejar eventos

Casos de uso:

Seguridad: Detectar inmediatamente cuando una puerta o ventana se abre (sensor magnético).

Interacción: Capturar la pulsación de un botón físico de pánico o configuración.

Eficiencia: Despertar al microcontrolador de un modo de "sueño profundo" (Deep Sleep) cuando un sensor de movimiento PIR detecta a una persona.

Ejemplo:

Imagina un dispositivo IoT con un botón que, al presionarse, debe encender un LED y enviar una alerta por red. Usaremos una interrupción para detectar la pulsación del botón de forma reactiva.

```
import machine
import time

# Configuración de pines
# Pin 0 como entrada con resistencia de pull-up para el botón
boton = machine.Pin(0, machine.Pin.IN, machine.Pin.PULL_UP)
# Pin 2 como salida para el LED
led = machine.Pin(2, machine.Pin.OUT)

# Variable global para indicar si el botón fue presionado (bandera)
boton_presionado = False

# Definición de la función manejadora de la interrupción (callback)
def manejador_interrupcion(pin):
    # Usamos la palabra clave 'global' para modificar la variable fuera de la función
    global boton_presionado
    boton_presionado = True
    # NOTA: No hacemos print ni lógica compleja aquí. Solo levantamos la bandera.

# Configuración de la interrupción en el pin del botón
# Se dispara en el flanco de bajada (cuando el botón conecta a tierra al presionarse)
boton.irq(trigger=machine.Pin.IRQ_FALLING, handler=manejador_interrupcion)

print("Esperando pulsación de botón...")

# Bucle principal (simulando otras tareas del dispositivo IoT)
while True:
    if boton_presionado:
        print("¡Evento de botón detectado!")
        # Manejamos la lógica compleja aquí, en el flujo principal, no en la interrupción
        led.on()      # Encendemos el LED
        print("Enviando alerta IoT por red...")
        time.sleep(2) # Simulamos una tarea larga, como una petición de red
        led.off()     # Apagamos el LED
        print("Alerta enviada. Volviendo a esperar.")

        # Reseteamos la bandera para esperar la próxima pulsación
        boton_presionado = False

    # Simulamos que el dispositivo hace otras cosas, como leer sensores
    # print("Leyendo sensores...", end='\r')
    time.sleep(0.1) # Pequeña pausa para no saturar la CPU
```

15. Funciones para manejar eventos

El Evento Temporal: Timer.init() (Temporizadores de Hardware):

Esta función es el reemplazo absoluto de `time.sleep()`. Cuando usamos `sleep()`, el microcontrolador queda "congelado" y sordo a otros eventos. `Timer.init()` permite programar un cronómetro interno en el chip que ejecuta una función cuando el tiempo se acaba, permitiendo que el bucle principal siga corriendo.

Conceptos a dominar:

period: El tiempo en milisegundos (ms). Ejemplo: 1000 = 1 segundo.

mode:

Timer.PERIODIC: Se repite en bucle infinito.

Timer.ONE_SHOT: Se ejecuta una sola vez y se destruye.

callback: La función que se ejecutará cuando el cronómetro llegue a cero.

15. Funciones para manejar eventos

Casos de uso

Telemetría: Leer un sensor de temperatura y enviar el dato a la nube exactamente cada 10 minutos, sin interrumpir la lectura de botones.

Indicadores de estado: Hacer parpadear un LED continuamente para indicar que el dispositivo está conectado al WiFi (Heartbeat).

Timeouts: Apagar una pantalla OLED automáticamente 30 segundos después de la última interacción del usuario.

Ejemplo:

Este ejemplo muestra cómo un dispositivo puede hacer parpadear un LED de forma autónoma mientras el bucle principal está ocupado haciendo otra cosa.

```
1  from machine import Pin, Timer
2  import time
3
4  # Configuración
5  led_estado = Pin(2, Pin.OUT)
6  temporizador = Timer(0) # Usamos el Timer 0 del hardware
7
8  # 1. Definimos el callback (La acción a ejecutar)
9  def parpadear_led(timer):
10     # Invertimos el estado actual del LED (si está en 1 pasa a 0, y viceversa)
11     led_estado.value(not led_estado.value())
12
13     # 2. Inicializamos el evento temporal
14     # Cada 500ms, de forma periódica, ejecutará 'parpadear_led'
15     temporizador.init(period=500, mode=Timer.PERIODIC, callback=parpadear_led)
16
17     # Bucle principal
18     while True:
19         print("El procesador está libre haciendo otras tareas pesadas...")
20         # Simulamos una tarea que toma mucho tiempo.
21         # El LED seguirá parpadeando en segundo plano gracias al Timer.
22         time.sleep(3)
```

Referencias bibliográficas alineadas con el Syllabus oficial

Libros y Textos Oficiales

- Hinojosa Gutiérrez, Á. (2023). Python: Paso a paso. RA-MA.
- Guzdial, M. J., & Ericson, B. (2021). Introducción a la computación y programación con Python (3.ª ed.). Pearson Educación.
- Singh, M., Kumar, B., & Arora, M. (2025). Microprogramación en Python: Guía completa para crear soluciones IoT. Ediciones Nuestro Conocimiento.
- Roy, T. P. (2024). Introducción a Python para internet de las cosas (IoT). Ediciones Nuestro Conocimiento.
- Peco, José. (2018). IoT con Raspberry Pi (2da ed.). CreateSpace Independent Publishing.
- Ramalho, Luciano. (2015). Fluent Python: Clear, Concise, and Effective Programming. O'Reilly Media.